

Comprehensive Results Report

Project: Build a Data Mart in SQL

Author: Alejandro Moral Aranda Date: August 17, 2025

Contents

1	Phase 1: Requirements Analysis and Initial Design	2
1.1	Summary	2
1.2	Document Delivery Requirements	2
1.3	Slides	4
2	Phase 2: Database Design and Implementation	7
2.1	Summary	7
2.2	Document Delivery Requirements: Development Phase	7
2.3	Slides	8
3	Phase 3: Testing, Finalization, and Analysis	10
3.1	Summary	10
3.2	Document Delivery Requirements	11
3.3	Slides	12

Abstract

This document presents a comprehensive overview of all results from each phase of the project Build a Data Mart in SQL. Included are summaries, analysis, slides, and screenshots produced throughout the three phases, demonstrating both the technical and presentation aspects of the work.

1 Phase 1: Requirements Analysis and Initial Design

1.1 Summary

In the first phase, I focused on understanding what the database for an Airbnb-like platform needed to do. The main goal was to identify all the different types of users, what actions they could perform, and what information the system should store and manage.

To do this, I wrote a requirements specification document that explained the roles in the system (like guests, hosts, and admins), the tasks each role can do, and the data and features needed to support these actions. I kept this document clear, short, and easy to read.

Next, I created an Entity Relationship Model (ERM) based on the requirements. This ERM showed the main entities, their attributes, and how they are connected. I included special connections like ternary relationships (joining three tables) and recursive relationships, to cover more complex scenarios. I also made sure to define key fields and explain how the entities are related.

Throughout this phase, I checked my work against the requirements and used feedback from tutorials and discussions to make improvements. At the end, I produced a clear design that will help guide the rest of the project.

1.2 Document Delivery Requirements

1. Requirements Specification Document See Figure 1 ✓

- Maximum length: 2 pages, 12pt font
- Must include detailed requirements analysis addressing:
 - Definition of user roles (person/user groups)
 - Description of actions performed by each role
 - Identification of required data and functions

2. Entity Relationship Model (ERM) Diagram See Figure 2 ✓

- Minimum 20 entities
- 2–3 ternary (triple) relationships
- Include any relevant recursive relationships
- Assign suitable attributes to each entity and mark key attributes
- Specify all cardinality requirements using a clear notation (e.g., Chen notation)

3. Data Dictionary See Figure 4 ✓

- Short description of all attributes
- Indicate data types for each attribute

4. **Abstract** See Figure 3 ✓

- Brief (approx. half-page) description of the work performed in this phase
- Include problem statement, solution approach, and main steps

1.3 Slides

Build a Data Mart in SQL (DLBDSPBDM01)

Conception Phase – Requirements Specification

Author: Alejandro Moral Aranda

Date: April 17, 2025

1 Introduction

This project designs a small Airbnb-style database with 22 tables. Two tables act as *triple links* that connect three different tables at once (`dispute_resolution` and `referral_rewards`). Another table, `admin_users`, points back to itself so we can store who manages whom (a *recursive* link). Together, these tables cover sign-up, property listings, bookings, payments, reviews, referral bonuses, and dispute handling.

2 Roles and User Groups

- **Guest**
Saved first in `user_profiles` and then in `guests`. Can book homes, write reviews, and earn referral credit.
- **Host**
Also starts in `user_profiles`, then `hosts`. Owns many `property_listings` plus calendars, prices, pictures, and rules.
- **Administrator**
Lives in `admin_users`. May supervise other admins, approve referral payouts, and close support tickets.

3 Actions and Processes

1. Guest Actions:

- Search homes by date, price, and amenity.
- Book stays, pay, or cancel if needed.
- Leave reviews for homes and hosts.
- Share referral codes and get paid when friends join.

2. Host Actions:

- Add or edit listings, photos, rules, and prices.
- Mark available dates and nightly rates.
- Chat with guests and resolve disputes.

Figure 1: Phase 1 - Slide 2: Requirement Specifications

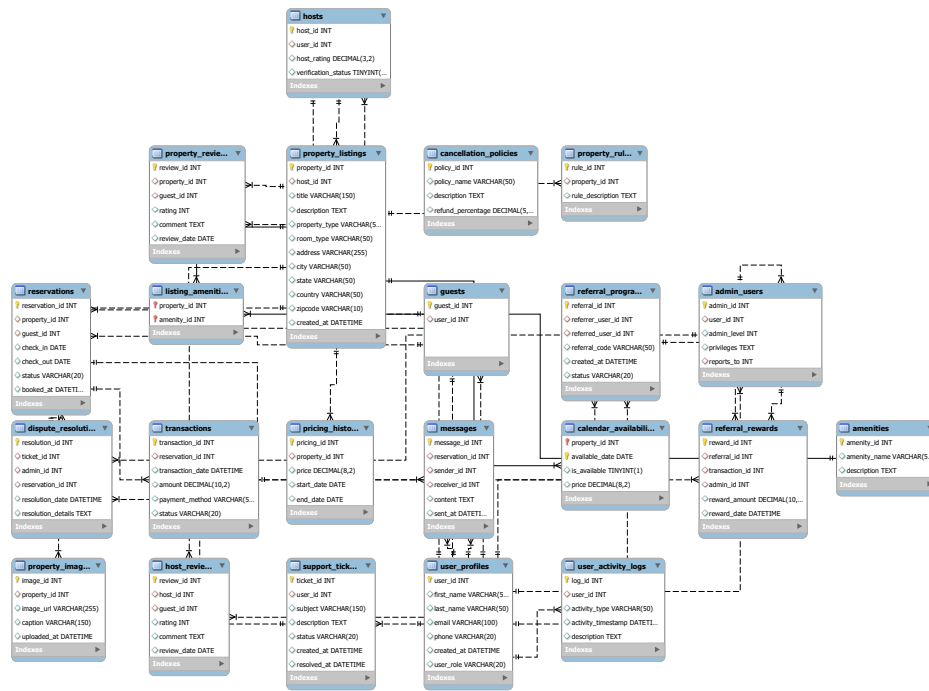


Figure 2: Phase 1 - Slide 1: ERM Overview

Build a Data Mart in SQL (DLBDSPBDM01)

Conception Phase – Abstract

Alejandro Moral Aranda

April 18, 2025

This abstract sums up the first phase of an Airbnb-like data-mart project. The goal is to keep every step of a short-term rental—sign-up, listing, booking, payment, chat, reviews, referrals, and disputes—in a format that is easy to analyse.

We designed a relational model with 22 tables:

- Two ternary tables store three-way facts in one row (`dispute_resolution` and `referral_rewards`).
- One recursive table (`admin_users`) keeps the admin hierarchy.

Every table has a primary key. Foreign keys link related tables. Only when a single column cannot identify a row, we use a composite key.

The schema works well for star-schema roll-ups, avoids update problems, and keeps nightly price history for time-series reports.

Steps in this phase

1. **Gather needs:** outlined roles (guest, host, admin) and core flows.
2. **Draw ER model:** listed entities and links; added two ternary and one recursive relationship.
3. **Build logical schema:** mapped the ER model to 22 SQL tables with data types and keys.
4. **Create database:** wrote and ran the `CREATE TABLE` script in MySQL Workbench.
5. **Check design:** turned on foreign-key checks and generated an EER diagram to verify links.

Figure 3: Phase 1 - Slide 2: Conception Phase Abstract

Data Dictionary

Table: user_profiles

Field Name	Data Type	Length	Description
user_id	INT	–	Unique identifier for each user (PK).
first_name	VARCHAR	50	First name of the user.
last_name	VARCHAR	50	Last name of the user.
email	VARCHAR	100	Email address for login and notifications.
phone	VARCHAR	20	Contact phone number of the user.
created_at	DATETIME	–	Timestamp when the profile was created.
user_role	VARCHAR	20	Role of the user (guest, host, admin).

Table: hosts

host_id	INT	–	Unique identifier for each host (PK).
user_id	INT	–	FK to user_profiles.user_id.
host_rating	DECIMAL	5,2	Average rating of the host.
verification_status	BOOLEAN	–	Whether the host is verified.

Table: guests

guest_id	INT	–	Unique identifier for each guest (PK).
user_id	INT	–	FK to user_profiles.user_id.

Table: admin_users

admin_id	INT	–	Unique identifier for each admin (PK).
user_id	INT	–	FK to user_profiles.user_id.
admin_level	INT	–	Permission level (e.g. 1=standard, 2=super).
privileges	TEXT	–	Detailed privileges (e.g. JSON).
reports_to	INT	–	FK to supervising admin_users.admin_id (recursive).

Table: property_listings

property_id	INT	–	Unique identifier for each property (PK).
host_id	INT	–	FK to hosts.host_id.
title	VARCHAR	150	Title of the listing.
description	TEXT	–	Detailed description of the property.
property_type	VARCHAR	50	Type of property (e.g. apartment, house).
room_type	VARCHAR	50	Type of room (entire home, private room).
address	VARCHAR	255	Street address of the property.
city	VARCHAR	50	City where the property is located.
state	VARCHAR	50	State or region.
country	VARCHAR	50	Country.
zipcode	VARCHAR	10	Postal code.
created_at	DATETIME	–	Timestamp when the listing was created.

Figure 4: Data Dictionary

2 Phase 2: Database Design and Implementation

2.1 Summary

In the second phase, I began the practical implementation of the Airbnb database system. Following the design from the ER diagram, I created all required tables and relationships using SQL. Each SQL statement was carefully documented, explaining its purpose and how it fits into the overall design.

I made sure that every table had at least 20 sample entries to test the structure and functionality. For quality assurance, I included test cases that checked whether the database worked as described in the ER model. These test cases were run in the database management system, and I took screenshots to show the results.

The work from this phase was presented in a detailed PDF of at least 20 slides, with each slide documenting one entity's SQL statement, its test case, and the screenshot of the result. This made the implementation process clear and easy to follow.

2.2 Document Delivery Requirements: Development Phase

1. SQL Database File: Table Creation, Data fill-in 5 ✓ and see 6 ✓

- Contains all SQL statements for table creation and relationship setup, as outlined in the ER diagram.
- Each SQL statement is clearly documented with comments explaining its purpose.
- Each table includes a minimum of 20 sample entries.

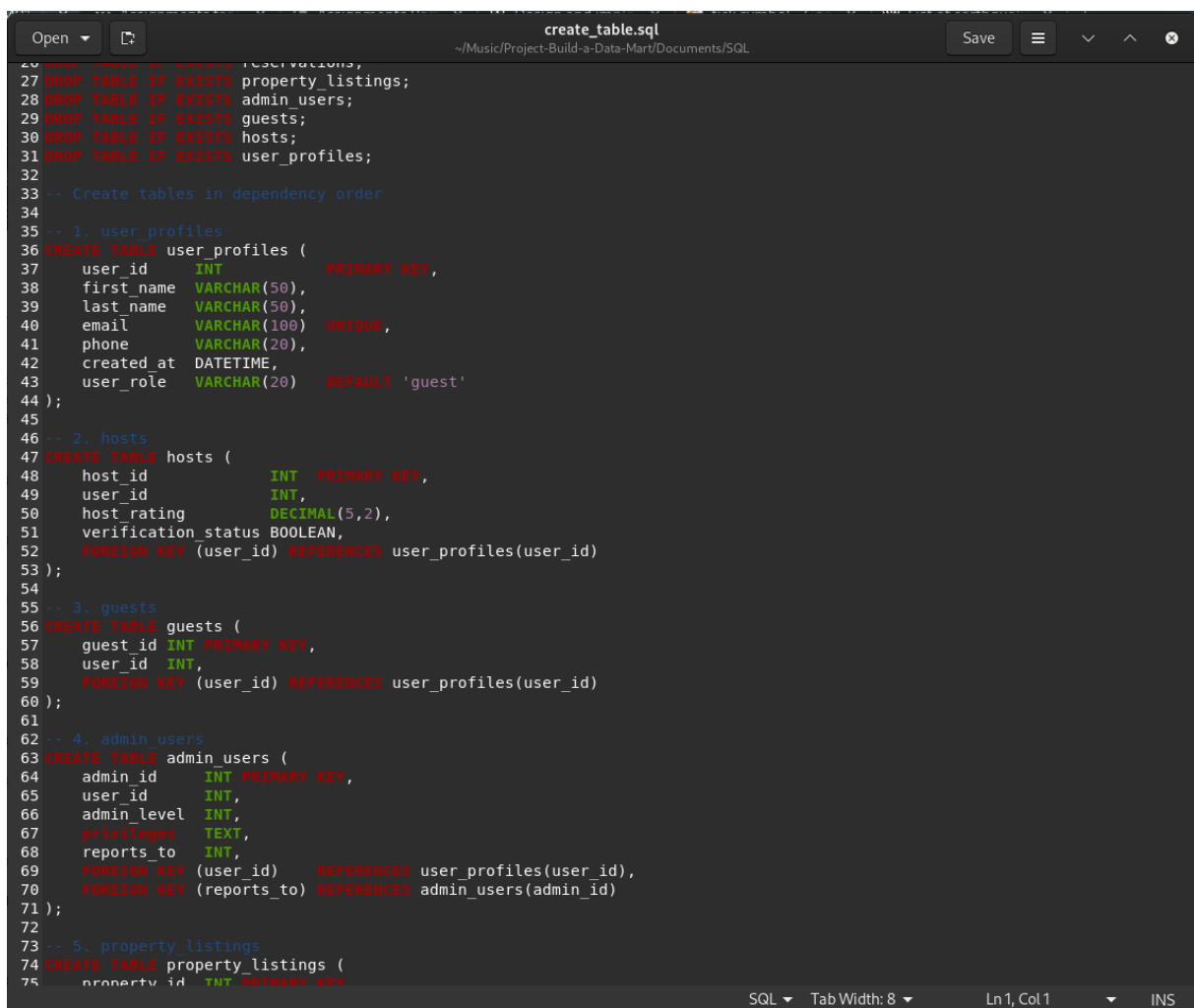
2. Test Cases see 7 ✓

- At least one test case for the database, verifying that the implementation matches the ER model.
- SQL statements for each test case included in the documentation.

3. Composite Presentation PDF see 8 ✓

- At least 20 slides, covering all entities and relationships from the ER model.
- Each slide documents the SQL statement for an entity, the relevant test case, and a screenshot showing the result in the database management system.

2.3 Slides



```

26 DROP TABLE IF EXISTS reservations;
27 DROP TABLE IF EXISTS property_listings;
28 DROP TABLE IF EXISTS admin_users;
29 DROP TABLE IF EXISTS guests;
30 DROP TABLE IF EXISTS hosts;
31 DROP TABLE IF EXISTS user_profiles;
32
33 -- Create tables in dependency order
34
35 -- 1. user profiles
36 CREATE TABLE user_profiles (
37     user_id INT PRIMARY KEY,
38     first_name VARCHAR(50),
39     last_name VARCHAR(50),
40     email VARCHAR(100) UNIQUE,
41     phone VARCHAR(20),
42     created_at DATETIME,
43     user_role VARCHAR(20) DEFAULT 'guest'
44 );
45
46 -- 2. hosts
47 CREATE TABLE hosts (
48     host_id INT PRIMARY KEY,
49     user_id INT,
50     host_rating DECIMAL(5,2),
51     verification_status BOOLEAN,
52     FOREIGN KEY (user_id) REFERENCES user_profiles(user_id)
53 );
54
55 -- 3. guests
56 CREATE TABLE guests (
57     guest_id INT PRIMARY KEY,
58     user_id INT,
59     FOREIGN KEY (user_id) REFERENCES user_profiles(user_id)
60 );
61
62 -- 4. admin users
63 CREATE TABLE admin_users (
64     admin_id INT PRIMARY KEY,
65     user_id INT,
66     admin_level INT,
67     privileges TEXT,
68     reports_to INT,
69     FOREIGN KEY (user_id) REFERENCES user_profiles(user_id),
70     FOREIGN KEY (reports_to) REFERENCES admin_users(admin_id)
71 );
72
73 -- 5. property listings
74 CREATE TABLE property_listings (
75     property_id INT PRIMARY KEY

```

Figure 5: SQL Database File: Table creation


```

data.sql
~/Music/Project-Build-a-Data-Mart/Documents/SQL

create_table.sql
data.sql

/*
2  SAMPLE DATA * MIN. 20 ROWS PER TABLE * FOR 22-TABLE AIRBNB MART
3  -----*/
4  USE airbnb;          -- change if your schema name differs
5  SET FOREIGN_KEY_CHECKS = 0;
6
7  /* 1. user_profiles (30 users) */
8  INSERT INTO user_profiles
9  (user_id, first_name, last_name, email, phone, created_at, user_role) VALUES
10 (1, 'Juan', 'Pérez', 'juan.perez@demo.com', '1000000001', NOW(), 'guest'),
11 (2, 'Beatriz', 'Sánchez', 'beatriz.sanchez@demo.com', '1000000002', NOW(), 'guest'),
12 (3, 'Carlos', 'Díaz', 'carlos.diaz@demo.com', '1000000003', NOW(), 'guest'),
13 (4, 'Diego', 'Romero', 'diego.romero@demo.com', '1000000004', NOW(), 'guest'),
14 (5, 'Elena', 'García', 'elena.garcia@demo.com', '1000000005', NOW(), 'guest'),
15 (6, 'Francisco', 'Martín', 'francisco.martin@demo.com', '1000000006', NOW(), 'host'),
16 (7, 'Gabriela', 'López', 'gabriela.lopez@demo.com', '1000000007', NOW(), 'host'),
17 (8, 'Hugo', 'Muñoz', 'hugo.munoz@demo.com', '1000000008', NOW(), 'host'),
18 (9, 'Inés', 'Costa', 'ines.costa@demo.com', '1000000009', NOW(), 'host'),
19 (10, 'Jorge', 'Nolasco', 'jorge.nolasco@demo.com', '1000000010', NOW(), 'host'),
20 (11, 'Karla', 'Yáñez', 'karla.yanez@demo.com', '1000000011', NOW(), 'guest'),
21 (12, 'Luis', 'Figueroa', 'luis.figueroa@demo.com', '1000000012', NOW(), 'guest'),
22 (13, 'María', 'Salinas', 'maria.salinas@demo.com', '1000000013', NOW(), 'guest'),
23 (14, 'Nicolás', 'Gómez', 'nicolas.gomez@demo.com', '1000000014', NOW(), 'guest'),
24 (15, 'Olivia', 'Chávez', 'olivia.chavez@demo.com', '1000000015', NOW(), 'guest'),
25 (16, 'Pablo', 'Silva', 'pablo.silva@demo.com', '1000000016', NOW(), 'guest'),
26 (17, 'Raquel', 'Grau', 'raquel.grau@demo.com', '1000000017', NOW(), 'guest'),
27 (18, 'Rodrigo', 'Cano', 'rodrigo.cano@demo.com', '1000000018', NOW(), 'guest'),
28 (19, 'Sara', 'López', 'sara.lopez@demo.com', '1000000019', NOW(), 'guest'),
29 (20, 'Tomás', 'Bravo', 'tomas.bravo@demo.com', '1000000020', NOW(), 'guest'),
30 (21, 'Úrsula', 'Soto', 'ursula.soto@demo.com', '1000000021', NOW(), 'guest'),
31 (22, 'Víctor', 'Navarro', 'victor.navarro@demo.com', '1000000022', NOW(), 'guest'),
32 (23, 'Walter', 'Huesca', 'walter.huesca@demo.com', '1000000023', NOW(), 'guest'),
33 (24, 'Ximena', 'Ibáñez', 'ximena.ibanez@demo.com', '1000000024', NOW(), 'guest'),
34 (25, 'Yago', 'Pascual', 'yago.pascual@demo.com', '1000000025', NOW(), 'guest'),
35 (26, 'Zacarías', 'Ibarra', 'zacarias.ibarra@demo.com', '1000000026', NOW(), 'admin'),
36 (27, 'Ana', 'Ruiz', 'ana.ruiz@demo.com', '1000000027', NOW(), 'admin'),
37 (28, 'Bruno', 'Segovia', 'bruno.segovia@demo.com', '1000000028', NOW(), 'admin'),
38 (29, 'Carmen', 'Luna', 'carmen.luna@demo.com', '1000000029', NOW(), 'admin'),
39 (30, 'Daniel', 'Olmedo', 'daniel.olmedo@demo.com', '1000000030', NOW(), 'admin');
40
41 /* 2. hosts (20) - users 6-25 */
42 INSERT INTO hosts (host_id, user_id, host_rating, verification_status) VALUES
43 (1, 6, 85.00, 1),
44 (2, 7, 90.00, 1),
45 (3, 8, 78.00, 0),
46 (4, 9, 88.00, 1),
47 (5, 10, 95.00, 1),

```

Figure 6: SQL Database File: Data Fill

```

1  --=====
2  --airbnb-test.sql -- integrity checks for the Airbnb data mart
3  --=====
4  USE airbnb;
5
6  /*
7   1. Volume check -- every table should report = 20 rows
8   */
9  SELECT 'user_profiles' AS table_name, COUNT(*) AS row_count FROM user_profiles
10 UNION ALL SELECT 'hosts' , COUNT(*) FROM hosts
11 UNION ALL SELECT 'guests' , COUNT(*) FROM guests
12 UNION ALL SELECT 'admin_users' , COUNT(*) FROM admin_users
13 UNION ALL SELECT 'property_listings' , COUNT(*) FROM property_listings
14 UNION ALL SELECT 'reservations' , COUNT(*) FROM reservations
15 UNION ALL SELECT 'property_reviews' , COUNT(*) FROM property_reviews
16 UNION ALL SELECT 'host_reviews' , COUNT(*) FROM host_reviews
17 UNION ALL SELECT 'property_images' , COUNT(*) FROM property_images
18 UNION ALL SELECT 'amenities' , COUNT(*) FROM amenities
19 UNION ALL SELECT 'listing_amenities' , COUNT(*) FROM listing_amenities
20 UNION ALL SELECT 'calendar_availability' , COUNT(*) FROM calendar_availability
21 UNION ALL SELECT 'pricing_history' , COUNT(*) FROM pricing_history
22 UNION ALL SELECT 'transactions' , COUNT(*) FROM transactions
23 UNION ALL SELECT 'messages' , COUNT(*) FROM messages
24 UNION ALL SELECT 'property_rules' , COUNT(*) FROM property_rules
25 UNION ALL SELECT 'cancellation_policies' , COUNT(*) FROM cancellation_policies
26 UNION ALL SELECT 'support_tickets' , COUNT(*) FROM support_tickets
27 UNION ALL SELECT 'user_activity_logs' , COUNT(*) FROM user_activity_logs
28 UNION ALL SELECT 'referral_programs' , COUNT(*) FROM referral_programs
29 UNION ALL SELECT 'dispute_resolution' , COUNT(*) FROM dispute_resolution
30 UNION ALL SELECT 'referral_rewards' , COUNT(*) FROM referral_rewards;
31
32 SELECT IF(MIN(row_count) >= 20, 'Volume check: OK', 'Volume check: FAIL')
33 FROM (
34   SELECT COUNT(*) AS row_count FROM user_profiles
35   UNION ALL SELECT COUNT(*) FROM hosts
36   UNION ALL SELECT COUNT(*) FROM guests
37   UNION ALL SELECT COUNT(*) FROM admin_users
38   UNION ALL SELECT COUNT(*) FROM property_listings
39   UNION ALL SELECT COUNT(*) FROM reservations
40   UNION ALL SELECT COUNT(*) FROM property_reviews
41   UNION ALL SELECT COUNT(*) FROM host_reviews
42   UNION ALL SELECT COUNT(*) FROM property_images
43   UNION ALL SELECT COUNT(*) FROM amenities
44   UNION ALL SELECT COUNT(*) FROM listing_amenities
45   UNION ALL SELECT COUNT(*) FROM calendar_availability
46   UNION ALL SELECT COUNT(*) FROM pricing_history
47   UNION ALL SELECT COUNT(*) FROM transactions

```

Figure 7: Test Cases

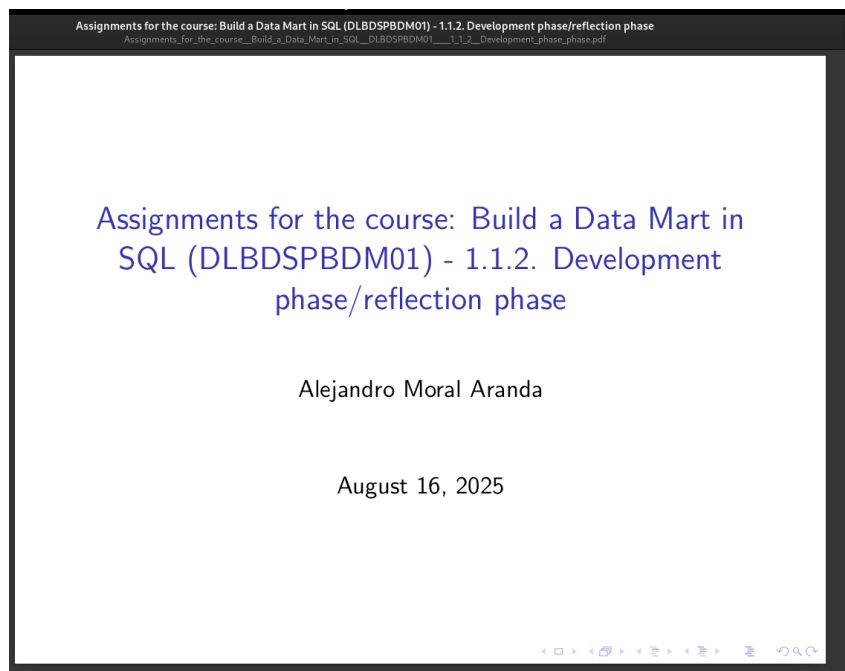


Figure 8: Composite Presentation

3 Phase 3: Testing, Finalization, and Analysis

3.1 Summary

In the final phase, I focused on refining and completing the Airbnb database management system. After receiving feedback from the tutor, I improved the system according to the

suggestions and checked the entire project for accuracy and completeness. The main goal was to finalize all elements, ensure thorough documentation, and prepare the project for submission.

As the last step, I wrote a two-page abstract summarizing the database's functionality and key metadata, such as the number of tables, total entries, and the overall database size. All required files, including SQL scripts, documentation, installation manual, and presentations, were organized and zipped into a single folder for submission. I also compiled a comprehensive PDF containing results from all three phases, along with slides and screenshots, and uploaded it to PebblePad.

3.2 Document Delivery Requirements

1. Two-Page Abstract (PDF) see 9 ✓

- Highlights and describes the database management system's functionality.
- Provides metadata: number of tables, total entries, and database size.

2. Complete Project Folder see 10 ✓

- Contains all project files: final SQL scripts, documentation, installation manual, slides, and screenshots.
- Folder is zipped into a single ZIP file for submission.

3. Comprehensive PDF Document (This document)

- Includes results, slides, and screenshots from all three phases.
- Uploaded separately to PebblePad.

3.3 Slides

Project: Build a Data Mart in SQL

Finalization Phase – Abstract

Author: Alejandro Moral Aranda

Date: August 17, 2025

Abstract

This project presents the design and implementation of a comprehensive relational database management system (DBMS) to support core Airbnb-like operations, including user management, property listings, bookings, payments, reviews, and support features. The system is structured around 22 normalized tables, ensuring robust data management, referential integrity, and extensibility for real-world deployment.

Database Design and Implementation: The schema employs the InnoDB storage engine with UTF-8 (utf8mb4) encoding to guarantee internationalization and data integrity. All tables are normalized and interlinked to support complex business logic and data associations.

System Metadata:

- **Total Tables:** 22
- **Total Records:** 440+ (minimum 20 entries per table)
- **Database Size:** Approximately 0.94 MB

Core Components:

- **User Management:** (user_profiles, hosts, guests, admin_users)
- **Property Management:** (property_listings, property_images, amenities, listing_amenities, property_rules, calendar_availability, pricing_history)
- **Transaction Management:** (reservations, transactions, cancellation_policies)
- **Support and Communication:** (property_reviews, host_reviews, messages, support_tickets, user_activity_logs)
- **Referral and Dispute Handling:** (referral_programs, referral_rewards, dispute_resolution)

Key Functionalities:

- Multi-role user management for hosts, guests, and administrators.
- Sophisticated reservation and payment processes, including platform-based payment retention and commission logic.
- Flexible property amenities and rules managed via many-to-many junction tables.
- Integrated review and messaging systems for user feedback and communication.
- Recursive and ternary relationships for referral tracking, administrative hierarchies, and dispute resolution.
- Advanced support for status tracking, cancellation policies, and user activity logging.

Figure 9: Phase 3 : Abstract

```

amoral-a@c2r10s2 ~/Music/Project-Build-a-Data-Mart/Documents
% ls -lR
.:
total 172
-rw-r--r-- 1 amoral-a 2025_luxembourg 168797 Aug 17 13:01 'Assignments Portfolio_DLBDSPBDM01.pdf'
drwxr-xr-x 2 amoral-a 2025_luxembourg 4096 Aug 16 22:33 'Conception Phase'
drwxr-xr-x 2 amoral-a 2025_luxembourg 117 Aug 17 12:57 'Development Phase'
drwxr-xr-x 2 amoral-a 2025_luxembourg 180 Aug 17 13:10 'Finalization phase'
drwxr-xr-x 2 amoral-a 2025_luxembourg 124 Aug 17 13:05 SQL

'./Conception Phase':
total 260
-rw-r--r-- 1 amoral-a 2025_luxembourg 54971 Aug 16 22:33 DLBDSPBDM01_Alejandro_MORAL_Aranda_09022025_92132071_Phase1_Abstract.pdf
-rw-r--r-- 1 amoral-a 2025_luxembourg 81343 Aug 16 22:33 DLBDSPBDM01_Alejandro_MORAL_Aranda_09022025_92132071_Phase1_Data_Dictionary.pdf
-rw-r--r-- 1 amoral-a 2025_luxembourg 42716 Aug 16 22:33 DLBDSPBDM01_Alejandro_MORAL_Aranda_09022025_92132071_Phase1_ERModel.pdf
-rw-r--r-- 1 amoral-a 2025_luxembourg 78663 Aug 16 22:33 DLBDSPBDM01_Alejandro_MORAL_Aranda_09022025_92132071_Phase1_Requirements_Specification_Document.pdf

'./Development Phase':
total 352
-rw-r--r-- 1 amoral-a 2025_luxembourg 357409 Aug 16 22:36 Assignments_for_the_course_Build_a_Data_Mart_in_SQL_DLBDSPBDM01_1_1_2_Development_phase_phase.pdf

'./Finalization phase':
total 1064
-rw-r--r-- 1 amoral-a 2025_luxembourg 1001010 Aug 17 13:10 DLBDSPBDM01_Alejandro_MORAL_Aranda_09022025_92132071_ComprehensivePDFdocument.pdf
-rw-r--r-- 1 amoral-a 2025_luxembourg 84150 Aug 17 12:57 'DLBDSPBDM01_Alejandro_MORAL_Aranda_09022025_92132071_Phase_Final_Abstract.pdf'

./SQL:
total 96
-rw-r--r-- 1 amoral-a 2025_luxembourg 739 Aug 16 22:42 cleandata.sql
-rw-r--r-- 1 amoral-a 2025_luxembourg 8076 Aug 16 22:42 create_table.sql
-rw-r--r-- 1 amoral-a 2025_luxembourg 24607 Aug 16 22:42 data.sql
-rw-r--r-- 1 amoral-a 2025_luxembourg 871 Aug 16 22:42 deletetables.sql
-rw-r--r-- 1 amoral-a 2025_luxembourg 3100 Aug 17 13:05 README.md
-rw-r--r-- 1 amoral-a 2025_luxembourg 6112 Aug 16 22:41 test.sql

```

Figure 10: Phase 3 : All the files